

思考ツールとしての型

－ 概念境界・代数的データ型・多相性・型による設計のためのノート －

Hiroki Funashima

Kindai University Technical College

Sunday, April 26, 2026

1 この文書の狙い

本ノートの目的は、型を単なるエラー検出の仕組みとしてではなく、思考ツールとして捉えることである。

プログラミングにおける型は、しばしば「安全性のための制約」として説明される。もちろん、それは重要な側面である。型によって、文字列を数値として扱う、存在しない値を不用意に参照する、誤った引数を関数に渡す、といったミスを検出しやすくなる。

しかし、型の役割はそれだけではない。特に Haskell のような静的型付き関数型言語では、型はプログラムの構造だけでなく、対象領域の概念構造を表すための道具になる。

本ノートでは、型を次のような観点から考える。

- 型は、概念の境界を明示する。
- 型は、関数が何を受け取り、何を返し、どこで失敗しうるかを表す。
- 代数的データ型は、対象が取りうる形を列挙する。
- 多相型は、特定の中身に依存しない構造を表す。
- 型クラスは、ある型が満たすべき能力や操作可能性を表す。
- `newtype` や `phantom type` は、意味の違いや状態の違いを型に刻む。
- スマートコンストラクタやモジュール境界は、型だけでは表せない不変条件を守る補助になる。

ただし、本ノートは動的型付けを否定するものではない。Python, Ruby, JavaScript, Lisp などの動的型付け言語は、探索的なプログラミング、柔軟なデータ処理、素早い試作において大きな力を持つ。本ノートで述べるのは、静的型、とくに Haskell の型が、別種の思考を支える道具として機能するということである。

2 型は制約ではなく問いの形式である

型 (type) は、よく「制約」として理解される。例えば、次の型を考える。

```
square :: Int -> Int
square x = x * x
```

この型は、`square` が `Int` を受け取り、`Int` を返すことを表している。`Int` は Haskell における固定幅整数型であり、任意精度整数を表す `Integer` とは区別される。確かに、これは制約である。`String` を渡すことはできないし、戻り値も `Int` でなければならない。

しかし、型を制約としてだけ読むと、その本質の一部を見落とす。型は同時に、「この関数は何を問うているのか」を表している。

```
square :: Int -> Int
```

これは、「整数を一つ与えたとき、整数を一つ返す変換」として読むことができる。もっと抽象的に言えば、型は関数が扱う対象の形式を示している。

例えば、次の型はどう読めるだろうか。

```
parseGameText :: String -> Either String Game
```

これは単に「文字列を受け取って何かを返す関数」ではない。この型は、次の情報を含んでいる。

- 入力 `String` である。
- 出力は成功または失敗を持つ。
- 失敗時には `String` のエラーメッセージが返る。
- 成功時には `Game` が返る。

つまり、この型は関数の仕様をかなり語っている。型は、プログラムが何を扱い、どのような可能性を持つのかを示す形式である。

この意味で、型は単なる制約ではなく、問いの形式である。

3 動的型付けを否定しない

型を思考ツールとして重視することは、動的型付けを否定することではない。

動的型付け言語には、明確な強みがある。例えば、Python では次のようなコードを素早く書ける。

```
def summarize(xs):
    return {
        "count": len(xs),
        "min": min(xs),
        "max": max(xs),
        "mean": sum(xs) / len(xs),
    }
```

このようなコードは、データの形を見ながら素早く試すのに向いている。探索的分析、可視化、Notebook 上での試行錯誤、API との連携などでは、動的型付けの柔軟性が強みになる。

一方、Haskell では、関数の型を先に考えることが多い。

```
summarize :: [Double] -> Either String Summary
```

この時点で、次のような問いが生じる。

- 空リストの場合はどうするのか。
- 失敗をどう表すのか。
- 結果は単なるタプルでよいのか、それとも専用の型を作るべきか。
- 平均、最小値、最大値は同じ概念として扱ってよいのか。

つまり、静的型付けは、書く前に考えることを促す。動的型付けは、動かしながら考えることを促す。

どちらが常に優れているという話ではない。むしろ、両者は異なる思考様式を支えている。

観点	動的型付け	静的型付け
強い局面	探索・試作・柔軟なデータ処理	設計・検証・長期的保守
思考の流れ	動かしながら考える	定義しながら考える
エラー検出	実行時に出やすい	コンパイル時に出やすい
概念境界	書き手の規律に依存しやすい	型として表しやすい
変更のしやすさ	初期は速い	構造が固まると強い

本ノートの立場は、動的型付けと静的型付けを優劣で比較することではない。重要なのは、型がどのような思考を支えるかを理解し、必要な場面で使い分けることである。

4 名前を付けることとしての型

型の最も基本的な役割の一つは、概念に名前を付けることである。

例えば、ゲーム理論のコードで戦略を文字列として扱うことはできる。

```
row :: String
row = "U"

col :: String
col = "L"
```

しかし、このままでは、行プレイヤーの戦略と列プレイヤーの戦略はどちらも `String` である。プログラム上は区別されない。

そこで、`newtype` を使う。

```
newtype RowStrategy = RowStrategy { unRowStrategy :: String }
    deriving (Eq, Ord, Show)

newtype ColStrategy = ColStrategy { unColStrategy :: String }
    deriving (Eq, Ord, Show)
```

この構文は、Haskell に不慣れだと少し読みにくいので、最初に分解しておく。

```
newtype RowStrategy = RowStrategy { unRowStrategy :: String }
```

この定義では、左辺の `RowStrategy` は新しく作る型の名前である。一方、右辺の `RowStrategy` は、その型の値を作るための値コンストラクタである。Haskell では、型名と値コンストラクタ名に同じ名前を使うことがよくある。

波括弧の中の

```
unRowStrategy :: String
```

は、レコード構文によるフィールド定義である。これは、`RowStrategy` の中身が `String` であり、その中身を取り出す関数として `unRowStrategy` が作られることを意味する。

したがって、次の式は文字列 `U` を中身に持つ `RowStrategy` 型の値である。

```
RowStrategy "U"
```

レコード構文を明示して、次のように書くこともできる。

```
RowStrategy { unRowStrategy = "U" }
```

また、内部の文字列は次のように取り出せる。

```
unRowStrategy (RowStrategy "U")
```

この式の結果は `U` である。つまり、`newtype` は、実行時にはほぼ中身の型と同じように扱える一方で、型検査上は別の型として扱われる。そのため、`String` と `RowStrategy` を区別できる。

また、末尾の

```
deriving (Eq, Ord, Show)
```

は、標準的な型クラスのインスタンスを自動的に導出する指定である。ここでは、次の意味を持つ。

- Eq によって、値同士を == で比較できる。
- Ord によって、値同士に順序を付けられる。
- Show によって、値を文字列として表示できる。

例えば、Show を導出しておくと、GHCi で値を表示したり、print で出力したりしやすくなる。deriving は、手で毎回比較関数や表示関数を書く代わりに、Haskell コンパイラに自然な実装を自動生成してもらう仕組みだと考えればよい。

これにより、実行時の中身はどちらも文字列であっても、型としては別物になる。RowStrategy を要求する関数に ColStrategy を渡すことはできない。

これは単なる安全性のためではない。ここで重要なのは、コード上に概念の境界が現れることである。

```
payoffAt :: Game -> RowStrategy -> ColStrategy -> Maybe Payoff
```

この型を見るだけで、payoffAt がゲーム、行戦略、列戦略を受け取り、利得が存在すれば返す関数だと分かる。もしこれが次のような型ならどうだろうか。

```
payoffAt :: Game -> String -> String -> Maybe Payoff
```

動くかもしれない。しかし、二つの String が何を意味するのかは、型だけからは分からない。

型に名前を付けることは、対象を分節化することである。これは、数理モデルや研究コードにおいて非常に重要である。

5 type と newtype の違い

Haskell では、型に名前を付ける方法として `type` と `newtype` がある。両者は似ているが、意味は異なる。

`type` は型の別名を作る。

```
type ParseResult = Either String Game
```

これは、`ParseResult` が `Either String Game` の別名であることを意味する。新しい型を作っているわけではない。

一方、`newtype` は新しい型を作る。

```
newtype Probability = Probability { unProbability :: Double }
```

この場合、`Probability` は `Double` とは別の型である。したがって、`Double` を要求する関数に `Probability` をそのまま渡すことはできないし、その逆もできない。

大まかに言えば、`type` は読みやすさのための名前付けであり、`newtype` は概念上の区別を型として作るための道具である。

構文	意味	用途
<code>type</code>	既存型への別名	読みやすさ、長い型の短縮
<code>newtype</code>	新しい型の定義	概念上の区別、誤用防止

この違いは、型を思考ツールとして使う上で重要である。単に名前を短くしたいなら `type` でよい。しかし、「同じ表現だが意味が違う」ものを区別したいなら、`newtype` を使う方がよい。

6 代数的データ型 (ADT, Algebraic Data Type)：対象が取りうる形を列挙する

Haskell の代数的データ型は、思考ツールとして非常に強力である。なぜなら、対象が取りうる形を明示的に列挙できるからである。

例えば、状態遷移を考える。

```
data ProcessState
  = NotStarted
  | Running Int
  | Finished
  | Failed String
deriving (Eq, Show)
```

この型は、プロセスの状態が次のいずれかであることを表している。

- まだ始まっていない。
- 実行中であり、進捗を表す整数を持つ。
- 完了している。
- 失敗しており、エラーメッセージを持つ。

これは、状態を単なる文字列や整数フラグで表す場合とは異なる。

```
-- 状態を文字列で表す例
state :: String
state = "running"
```

文字列でも表現はできる。しかし、可能な状態の集合は型には現れない。誤って "runnning" と書いても、文字列としては成立してしまう。

代数的データ型では、可能な形を閉じた集合として定義できる。そのため、状態の分類が型の中に現れる。

さらに、パターンマッチによって、状態ごとの処理を明示できる。

```
describeState :: ProcessState -> String
describeState NotStarted    = "not started"
describeState (Running n)   = "running: " ++ show n
describeState Finished      = "finished"
describeState (Failed err)  = "failed: " ++ err
```

ここで重要なのは、型が「状態とは何か」を記述していることである。代数的データ型は、対象領域の小さな分類体系をコード上に作る道具である。

7 直積と直和として考える

代数的データ型は、直積型 (direct product type) と直和型 (direct sum type) の組み合わせとして理解できる。

例えば、次の型を考える。

```
data Payoff = Payoff
  { payoffP1 :: Double
  , payoffP2 :: Double
  }
  deriving (Eq, Show)
```

これは、プレイヤー 1 の利得とプレイヤー 2 の利得を両方持つ。つまり、Double と Double の組である。このような構造は直積的である。

一方、次の型は直和的である。

```
data ParseResult
  = ParseSuccess Game
  | ParseFailure String
  deriving (Show)
```

これは、成功か失敗かのどちらかである。具体的には、成功なら Game を持ち、失敗なら String を持つ。

上の独自定義の代わりに、Haskell の標準的な型である Either を使うこともできる。

```
type ParseResultE = Either String Game
```

ここで ParseResultE は Either String Game の別名である。同じモジュール内で data ParseResult = ... と type ParseResult = ... を同時に定義することはできないため、ここでは別名として ParseResultE と書いている。

直積は「A かつ B」であり、直和は「A または B」である。この二つを組み合わせることで、かなり多くの概念構造を表現できる。

例えば、ゲームのセルは次のように表せる。

```
data Cell = Cell
  { cellRowStrategy :: RowStrategy
  , cellColStrategy :: ColStrategy
  , cellPayoff      :: Payoff
  }
  deriving (Eq, Show)
```

これは、行戦略、列戦略、利得をすべて持つ直積的な構造である。

一方、ゲームの解析結果は直和的に表せる。

```
data AnalysisResult
  = PureEquilibria [PureNashEquilibrium]
  | MixedEquilibria [MixedNashEquilibrium]
  | NoPureEquilibriumFound
  | AnalysisError String
deriving (Show)
```

ここでは、NoPureEquilibriumFound としている。有限標準形ゲームでは混合戦略 Nash 均衡の存在定理があるため、単に NoEquilibriumFound と書くと誤解を招く可能性がある。純粋戦略均衡が見つからないこと、あるいは計算上の失敗を表すことは、より明示的に区別した方がよい。

このように、代数的データ型は、対象の構造を「何を持つか」と「どの形を取りうるか」に分けて考える道具になる。

8 Maybe と Either : 不在と失敗を型に出す

型を思考ツールとして見るとき、Maybe と Either は非常に重要である。

Maybe a は、「a が存在するかもしれないし、存在しないかもしれない」ことを表す。

```
safeHead :: [a] -> Maybe a
safeHead []    = Nothing
safeHead (x:_) = Just x
```

この型を見れば、空リストの場合に結果が存在しないことが分かる。

一方、Either e a は、本来は「e または a」という直和型である。実用上は、Haskell では慣習的に Left を失敗、Right を成功として使うことが多い。

```
safeDiv :: Double -> Double -> Either String Double
safeDiv _ 0 = Left "division by zero"
safeDiv x y = Right (x / y)
```

ここでは、失敗理由を String として返している。

Maybe と Either の違いは、失敗理由を持つかどうかである。

型	意味	使いどころ
Maybe a	値があるかもしれない	不在だけを表したい場合
Either e a	失敗または成功	失敗理由を持たせたい場合

重要なのは、これらが関数の型に現れることである。

```
lookupPayoff :: Game -> RowStrategy -> ColStrategy -> Maybe Payoff
parseGame    :: String -> Either String Game
```

一つ目は、利得が存在しない可能性を示している。二つ目は、パースが失敗しうることを示している。

型を見ることで、関数がどのようなリスクや可能性を持つのかが分かる。これは、型を仕様として読むということである。

9 多相型 (polymorphic type) : 中身に依存しない構造を表す

多相型もまた、思考ツールとして重要である。多相型は、特定の型に依存しない構造を表す。

例えば、次の関数を考える。

```
identity :: a -> a
identity x = x
```

この型は、任意の型 a について、 a を受け取り、同じ a を返すことを表す。この関数は、 a の中身について何も知らない。したがって、できることはほとんどない。実質的に、受け取ったものをそのまま返すしかない。

これは制約であると同時に、強い情報でもある。多相型は、「この関数は中身に依存していない」という事実を表す。

例えば、次の関数を考える。

```
mapMaybe :: (a -> Maybe b) -> [a] -> [b]
mapMaybe _ [] = []
mapMaybe f (x:xs) =
  case f x of
    Nothing -> mapMaybe f xs
    Just y   -> y : mapMaybe f xs
```

この型は、 a や b の具体的な中身に依存していない。関心があるのは、各要素を $\text{Maybe } b$ に変換し、成功したものだけを集めるという構造である。

多相型は、具体的な対象から一段上がった構造を表す。そのため、型を見るだけで、関数が何に依存し、何に依存していないかが分かる。

10 多相性は抽象の範囲を示す

多相型は、抽象化の範囲を示す。

例えば、次の型を試してみる。

```
length :: [a] -> Int
```

これは、任意の型 a のリストを受け取り、長さを返す関数である。この関数は、リストの中身が整数であろうと、文字列であろうと、ゲームであろうと、戦略であろうと関係ない。必要なのは「リストである」という構造だけである。

一方、次の型はどうだろうか。

```
sum :: Num a => [a] -> a
```

これは、任意の a ではなく、`Num` 型クラスのインスタンスである a に限られる。つまり、足し算ができるもののリストだけを扱う。

このように、型変数と型クラス制約は、関数が何を必要としているかを示す。

型	読み方	必要な情報
<code>[a] -> Int</code>	任意の要素型のリストから長さを返す	リスト構造だけ
<code>Num a => [a] -> a</code>	数値的に足せる要素のリストを合計する	足し算とゼロ
<code>Ord a => [a] -> a</code>	順序付けできる要素から代表値を選ぶ	比較可能性
<code>Show a => a -> String</code>	表示可能な値を文字列にする	表示方法

多相性は、抽象化の範囲を可視化する。関数が具体的な値の中身を見ているのか、それとも構造だけを見ているのか。型はその違いを教えてくれる。

11 型クラス：能力を制約として表す

型クラスは、型が持つべき能力を表す仕組みである。

例えば、等値比較できる型には `Eq` がある。

```
(==) :: Eq a => a -> a -> Bool
```

この型は、`a` が `Eq` のインスタンスであるなら、二つの `a` を比較して `Bool` を返せることを表している。

順序比較なら `Ord` である。

```
maximum :: Ord a => [a] -> a
```

これは、順序付けできる値のリストから最大値を返す関数である。

型クラス制約は、「この関数は対象について何を仮定しているのか」を示す。例えば、次の型を考える。

```
best :: Ord score => (a -> score) -> [a] -> Maybe a
```

この関数は、各 `a` にスコアを与える関数と、`a` のリストを受け取り、最良の `a` を返すかもしれない。ここで必要なのは、スコアが順序付けできることだけである。`a` 自体が何であるかは問わない。

このように、型クラスは必要な能力だけを明示する。これは、設計上非常に重要である。なぜなら、関数が要求する前提を最小限にできるからである。

12 型は設計上の過剰な仮定を見つける

型を書いていると、関数が必要以上の仮定を置いていることに気づくことがある。

例えば、次のような関数を考える。

```
chooseBestInt :: (a -> Int) -> [a] -> Maybe a
```

これは、候補に整数スコアを与え、最良の候補を選ぶ関数である。しかし、本当にスコアは `Int` でなければならないのだろうか。

必要なのは、スコアを比較できることだけかもしれない。その場合、型は次のように一般化できる。

```
chooseBest :: Ord score => (a -> score) -> [a] -> Maybe a
```

この型では、スコアは `Int` に限られない。`Double` でも、辞書式順序を持つタプルでも、独自に `Ord` を定義した型でもよい。

このように、型を見直すことで、設計上の過剰な仮定を外せる。逆に、一般化しすぎている場合にも型は警告を与えてくれる。

例えば、次の型は非常に一般的である。

```
process :: a -> a
```

この関数は任意の `a` に対して動かなければならない。そのため、`a` の中身を見たり、変換したりすることは基本的にできない。この型で本当に十分なのか、あるいは何らかの制約が必要なのかを考える必要がある。

型は、抽象化しすぎていないか、具体化しすぎていないかを考えるための道具である。

13 型駆動設計：型から実装へ

Haskell では、型を先に書き、その型に合う実装を考えることがよくある。これは型駆動開発 (type-driven development) と呼ばれることがある。

例えば、意思決定理論のモデルを考える。状態、行為、結果、効用があるとする。

```
newtype State = State { unState :: String }
    deriving (Eq, Ord, Show)

newtype Act = Act { unAct :: String }
    deriving (Eq, Ord, Show)

newtype Outcome = Outcome { unOutcome :: String }
    deriving (Eq, Ord, Show)

newtype Utility = Utility { unUtility :: Double }
    deriving (Eq, Ord, Show)
```

次に、行為と状態から結果を返す関数、結果から効用を返す関数を考える。

```
type Consequence = Act -> State -> Outcome
type UtilityFunction = Outcome -> Utility
```

状態に確率が付いているなら、期待効用は次のような型を持つ。

```
newtype Probability = Probability { unProbability :: Double }
    deriving (Eq, Ord, Show)

expectedUtility
    :: [(State, Probability)]
    -> Consequence
    -> UtilityFunction
    -> Act
    -> Utility
```

この型を書くだけで、実装前にかなり多くの設計が明らかになる。

- 状態には確率が付く。
- 行為と状態から結果が決まる。
- 結果から効用が決まる。
- 評価対象は一つの行為である。
- 返るものは効用である。

型は、実装の前に概念関係を整理するための下書きになる。この意味で、型は思考のスケッチである。

14 型で保証できることと保証できないこと

型は強力な思考ツールであるが、型を書けばすべての性質が自動的に保証されるわけではない。例えば、次のように `Probability` を定義したとする。

```
newtype Probability = Probability { unProbability :: Double }
deriving (Eq, Ord, Show)
```

この型によって、`Probability` と `Utility` や `Distance` を区別することはできる。しかし、`Probability` の中身が本当に 0 以上 1 以下であることまでは、この定義だけでは保証されない。

```
badProbability :: Probability
badProbability = Probability 1.7
```

これは型としては通ってしまう。したがって、「確率である」という概念名を型で与えることと、「確率として妥当な値である」ことを保証することは区別しなければならない。

妥当性まで保証したい場合は、コンストラクタを直接公開せず、スマートコンストラクタを用意する方法がある。

```
module Probability
( Probability
, mkProbability
, unProbability
) where

newtype Probability = Probability { unProbability :: Double }
deriving (Eq, Ord, Show)

mkProbability :: Double -> Maybe Probability
mkProbability p
  | 0 <= p && p <= 1 = Just (Probability p)
  | otherwise         = Nothing
```

このようにすれば、モジュール外部からは不正な `Probability` を直接作れない。型は概念の境界を示し、スマートコンストラクタは値の不変条件を守る。

この区別は重要である。型は多くの構造を明示できるが、すべての意味的性質を自動的に保証するわけではない。必要に応じて、コンストラクタの公開範囲、検証関数、スマートコンストラクタ、あるいはより強い型レベル設計を組み合わせる必要がある。

15 型穴：コンパイラと対話する

Haskell では、型穴を使ってコンパイラと対話しながら実装を進めることができる。

例えば、次のように書く。

```
expectedUtility
  :: [(State, Probability)]
  -> Consequence
  -> UtilityFunction
  -> Act
  -> Utility
expectedUtility belief consequence utility act = _
```

ここで `_` は型穴である。GHC は、この場所にどの型の式が必要かを教えてくれる。

型穴は、実装を怠けるためのものではない。むしろ、設計した型に対して、次に何を作ればよいかをコンパイラに尋ねるための道具である。

型を先に置き、型穴で不足部分を確認し、少しずつ実装を埋める。この作業は、型を介した対話的な思考である。

Python の REPL や Notebook が値を見ながら考えるための環境だとすれば、Haskell の型穴は型を見ながら考えるための環境である。

16 Phantom type：状態の違いを型に埋め込む

型は、値の形だけでなく、状態の違いを表すためにも使える。その一例が phantom type である。

例えば、解析前のゲームと解析済みのゲームを区別したいとする。ここでは、型レベルのタグとして Raw と Checked を用意する。

```
data Raw = Raw
data Checked = Checked

newtype Game state = Game { unGame :: String }
    deriving (Eq, Show)
```

ここで、state は値の中には現れない。しかし、型レベルでは Game Raw と Game Checked は別物である。このように、値の中に現れない型引数を使って状態や段階を区別する手法を phantom type と呼ぶ。

```
parseGame :: String -> Either String (Game Raw)
parseGame text = Right (Game text)

checkGame :: Game Raw -> Either String (Game Checked)
checkGame (Game text) = Right (Game text)

analyzeGame :: Game Checked -> AnalysisResult
analyzeGame _ = NoPureEquilibriumFound
```

この設計では、analyzeGame は Game Checked しか受け取れない。つまり、チェック前のゲームを誤って解析することを型で防ぎやすくなる。

これは状態遷移を型に埋め込む小さな例である。

```
parseGame    :: String -> Either String (Game Raw)
checkGame    :: Game Raw -> Either String (Game Checked)
analyzeGame  :: Game Checked -> AnalysisResult
```

この型の並びを見るだけで、処理の順序が分かる。型は、ワークフローの段階を表す道具にもなる。

17 GADT：構文と意味の対応を強める

より高度な話として、GADT を使うと、構文と意味の対応を型でより強く表すことができる。

例えば、単純な式言語を考える。整数式と真偽値式を混ぜて扱いたいが、意味のない式は作れないようにしたい。

```
{-# LANGUAGE GADTs #-}

data Expr a where
  IntLit  :: Int -> Expr Int
  BoolLit :: Bool -> Expr Bool
  Add     :: Expr Int -> Expr Int -> Expr Int
  Equal   :: Expr Int -> Expr Int -> Expr Bool
  If      :: Expr Bool -> Expr a -> Expr a -> Expr a
```

この型では、式の結果型が `Expr a` の `a` に現れる。

例えば、整数リテラルは `Expr Int` である。

```
IntLit 3 :: Expr Int
```

真偽値リテラルは `Expr Bool` である。

```
BoolLit True :: Expr Bool
```

加算は整数式同士にしか使えない。

```
Add :: Expr Int -> Expr Int -> Expr Int
```

したがって、次のような式は型が合わない。

```
-- Add (BoolLit True) (IntLit 3) は型エラーになる
```

評価関数は次のように書ける。

```
eval :: Expr a -> a
eval (IntLit n)    = n
eval (BoolLit b)   = b
eval (Add x y)     = eval x + eval y
eval (Equal x y)   = eval x == eval y
eval (If c t e)    = if eval c then eval t else eval e
```

ここでは、式の構文と評価結果の型が対応している。GADT は、DSL や形式言語、型付き構文木を扱うときに強力である。

ただし、常に GADT を使うべきという意味ではない。重要なのは、必要になったときに、構文上の制約や意味上の制約を型で表せるということである。

18 型はドキュメントか

型はドキュメントの一種である。しかし、通常のコメントとは違う。

コメントは、実装とずれることがある。

```
-- This function never fails.  
parseGame :: String -> Either String Game
```

このコメントは、型と矛盾している。型を見ると、`Either String Game` なので失敗しうることが分かる。

型はコンパイラによって検査される。そのため、型は機械的に確認されるドキュメントである。

もちろん、型だけで全てが分かるわけではない。例えば、次の型を見ても、具体的にどのような形式の文字列を受け付けるのかまでは分からない。

```
parseGame :: String -> Either String Game
```

そこには自然言語の説明や仕様書が必要である。しかし、型は少なくとも、入力、出力、失敗可能性、副作用、抽象化の範囲を示す。

この意味で、型は実装と同期する設計メモである。

19 型は研究メモにもなる

研究コードにおいて、型は研究メモとしても機能する。

例えば、ゲーム理論の標準形ゲームを次のように表す。

```
data Game = Game
  { rowStrategies :: [RowStrategy]
  , colStrategies :: [ColStrategy]
  , payoffAtCell  :: RowStrategy -> ColStrategy -> Maybe Payoff
  }
```

この型は、ゲームをどのような対象として見ているかを表している。戦略集合があり、行戦略と列戦略の組に対して利得がある。ただし、利得が存在しない可能性もある。

もし、利得行列が必ず完全であることを前提にするなら、型は変わる。

```
data CompleteGame = CompleteGame
  { rowStrategiesC :: [RowStrategy]
  , colStrategiesC :: [ColStrategy]
  , payoffAtCellC  :: RowStrategy -> ColStrategy -> Payoff
  }
```

この二つの型の違いは、研究上の前提の違いである。

- Game は部分的な利得表を許す。
- CompleteGame は利得が必ず定義されていることを仮定する。

このように、型は単なる実装上の都合ではなく、モデル化の前提を記録する。研究メモとして型を残しておくことは、自分が何を仮定していたかを後から確認する手がかりになる。

20 型による分解と論文構造

型による分解は、論文構造にも影響を与えることがある。

例えば、あるアルゴリズムを次のような関数群に分けたとする。

```
parseGame      :: String -> Either String Game
markResponses  :: Game -> MarkedGame
pureEquilibria :: MarkedGame -> [PureNashEquilibrium]
mixedCandidates :: Game -> [MixedCandidate]
verifyMixed    :: Game -> MixedCandidate -> Either String MixedNashEquilibrium
renderReport   :: AnalysisReport -> String
```

この型の並びを見ると、プログラムの処理段階が分かる。

- 入力を読む。
- best response を記録する。
- 純粋均衡を抽出する。
- 混合均衡候補を作る。
- 候補を検証する。
- レポートを表示する。

これは、そのまま論文や技術ノートの節構成にもなりうる。

型によってコードを分解すると、処理の意味単位が現れる。その意味単位は、説明の単位にもなる。

この意味で、型は実装だけでなく、文章構成の足場にもなる。

21 型を細かくしすぎる危険

型は便利だが、細かくしすぎればよいというものではない。

例えば、すべての値に別々の `newtype` を与えれば、たしかに区別は明確になる。しかし、あまりに細かく分けすぎると、コードが重くなり、実装の見通しが悪くなることもある。

```
newtype RowIndex = RowIndex Int
newtype ColumnIndex = ColumnIndex Int
newtype StrategyName = StrategyName String
newtype PlayerName = PlayerName String
newtype PayoffValue = PayoffValue Double
newtype UtilityValue = UtilityValue Double
```

このような設計が有効な場合もある。しかし、常に必要とは限らない。

型を導入する基準は、「そこに概念的な区別があり、その区別を間違えるリスクや意味があるか」である。

例えば、`Probability` と `Utility` は区別する価値が高い。両者はどちらも `Double` で表せるが、意味がまったく異なるからである。

一方、局所的な計算の中だけで使う一時的な数値にまで専用型を作る必要はないかもしれない。

型は思考ツールである。思考を助けるために使うべきであり、思考を妨げるほど細分化する必要はない。

22 抽象化しすぎる危険

Haskell に慣れてくると、抽象化を進めたいくなる。型クラス、多相性、GADT、型族、依存型風の設計など、多くの道具がある。

しかし、研究コードや教育用コードでは、抽象化しすぎることに注意が必要である。

例えば、均衡概念まで一般化しようとする、次のような設計を考えたい。ここでの `Equilibrium` は説明用の仮の型である。

```
{-# LANGUAGE TypeFamilies #-}

class GameLike g where
  type Strategy g
  type Payoff g
  type Equilibrium g
  equilibria :: g -> [Equilibrium g]
```

このような抽象化は、適切な段階では有効である。しかし、標準形ゲームについて考え始めた段階では、次のように具体的に書く方がよい場合も多い。

```
data NormalFormGame = NormalFormGame
  { rowStrategies :: [RowStrategy]
  , colStrategies :: [ColStrategy]
  , payoffMatrix  :: [[Payoff]]
  }
```

抽象化は、共通構造が見えてから行う方がよい。先に抽象化しすぎると、コードが理論の整理ではなく、抽象化そのものの実験になってしまう。

型は思考を助ける。しかし、過剰な型レベル設計は、思考の焦点をずらすこともある。

23 思考ツールとして型を使うための指針

型を思考ツールとして使うためには、次のような指針が役に立つ。

1. 同じ実行時表現でも、意味が違えば型を分ける。
2. 型の別名で十分なら `type`、概念上の区別が必要なら `newtype` を使う。
3. 対象が取りうる形が限られているなら、代数的データ型で表す。
4. 値が存在しない可能性があるなら、`Maybe` を使う。
5. 失敗理由を持たせたいなら、`Either` を使う。
6. 副作用があるなら、`IO` を型に出す。
7. 中身に依存しない処理なら、多相型で表す。
8. 必要な能力だけを型クラス制約として要求する。
9. 値の妥当性まで守りたいなら、スマートコンストラクタやモジュール境界を使う。
10. 処理段階の違いが重要なら、`phantom type` などで状態を分ける。
11. DSL や式言語では、必要に応じて `GADT` を検討する。
12. 型を細かくしすぎず、概念上意味のある境界に集中する。

これらは絶対的な規則ではない。しかし、型を単なるコンパイルのための記述ではなく、思考を整理するための道具として使うための目安になる。

24 まとめ

本ノートでは、型を思考ツールとして捉える見方を整理した。
要点は次の通りである。

1. 型は単なる制約ではなく、問いの形式である。
2. 動的型付けは探索や試作に強く、静的型付けは設計や長期的な構造化に強い。
3. `type` は型の別名を作り、`newtype` は概念上の区別を型として作る。
4. `newtype` は、同じ実行時表現を持つ概念を型として区別する。
5. 代数的データ型は、対象が取りうる形を明示する。
6. 直積型と直和型を組み合わせることで、概念構造を表現できる。
7. `Maybe` と `Either` は、不在や失敗を型に現す。
8. 多相型は、中身に依存しない構造を表す。
9. 型クラスは、必要な能力を制約として表す。
10. 型は概念境界を示すが、値の妥当性まで常に自動保証するわけではない。
11. `phantom type` や `GADT` は、状態や構文の意味をより強く型に反映できる。
12. 型は、研究コードの前提や処理段階を記録するメモにもなる。

型は、プログラムを狭くするためだけのものではない。むしろ、型は考える対象を分け、名前を付け、可能な形を列挙し、失敗や副作用を明示し、抽象化の範囲を示す。

この意味で、型は思考の道具である。特に Haskell では、型を書くことが、そのまま対象をどう理解しているかを表す行為になる。

動的型付けの柔軟性が探索を支えるように、静的型付けの明示性は構造化を支える。どちらが常に優れているという話ではない。重要なのは、いま自分が行っている作業が、探索なのか、整理なのか、形式化なのか、研究資産化なのかを見極めることである。

型は、整理と形式化の局面でとくに力を発揮する。そして、数理モデル、ゲーム理論、意思決定理論、状態遷移、パーサ、DSL のような領域では、型は単なるプログラミング上の補助ではなく、概念そのものを考えるための足場になる。

付録 A Appendix：型を読むための短い練習

最後に、いくつかの型を読み、その意味を考える練習を置く。

```
safeHead :: [a] -> Maybe a
```

これは、任意の型 `a` のリストを受け取り、先頭要素があれば返す関数である。空リストの場合は `Nothing` になる。

```
parseGame :: String -> Either String Game
```

これは、文字列をゲームとして解釈しようとする関数である。失敗すればエラーメッセージ、成功すれば `Game` を返す。

```
readGame :: FilePath -> IO (Either String Game)
```

これは、ファイルを読むという副作用を持ち、その結果としてゲームのパーズに成功または失敗する関数である。

```
chooseBest :: Ord score => (a -> score) -> [a] -> Maybe a
```

これは、候補にスコアを与える関数と候補リストを受け取り、最良の候補があれば返す関数である。スコアには順序が必要である。

```
transform :: (a -> b) -> [a] -> [b]
```

これは、a を b に変換する関数を使って、a のリストを b のリストに変換する関数である。これは map の型である。

```
mkProbability :: Double -> Maybe Probability
```

これは、Double を受け取り、妥当な確率であれば Probability を返す関数である。妥当でない場合は Nothing になる。

```
checkGame :: Game Raw -> Either String (Game Checked)
```

これは、未検査のゲームを受け取り、検査に成功すれば検査済みのゲームを返す関数である。失敗する場合はエラーメッセージを返す。

付録 B Appendix：型設計の短い指針

研究コードを書くとき、次の問いを立てると型設計がしやすい。

- この値は、既存の型で十分か、それとも概念名を与えるべきか。
- それは単なる別名でよいのか、それとも新しい型として区別すべきか。
- この関数は失敗しうるか。
- 失敗するなら、理由は必要か。
- この処理は副作用を持つか。
- このデータは、いくつかの形のうち一つを取るのか。
- この関数は、具体的な型に依存しているのか、それとも構造だけに依存しているのか。
- この型クラス制約は本当に必要か。
- この型で保証したいのは概念上の区別か、値の妥当性か。
- 値の妥当性を守るために、コンストラクタを隠す必要があるか。
- 処理の順序を型で表す価値があるか。
- 抽象化を今行うべきか、もう少し具体例を集めるべきか。

型設計は、一度で完成させるものではない。研究テーマと同じように、書きながら、読みながら、使いながら育てるものである。